

A PROJECT REPORT ON

UPI TRANSACTION FRAUD DETECTION SYSTEM

Domain: Machine Learning / FinTech Security

Submitted in partial fulfilment of the requirements for the award of the degree of

B.Sc. / B.E. / MCA – COMPUTER SCIENCE

Submitted by

STUDENT NAME

(Register No: XXXXXXXXX)

Under the guidance of

GUIDE NAME, M.Sc., M.Phil.,

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE

COLLEGE NAME

2025 – 2026

CONTENTS

CHAPTER	TITLE	PAGE
I	INTRODUCTION	1
	1.1 History of Domain	
	1.2 Overview of the Project	
	1.3 Motivation	
	1.4 Aim of the Project	
	1.5 Objectives	
	1.6 Literature Review	
II	SOFTWARE PROJECT PLAN	5
	2.1 Existing System	
	2.2 Proposed System	
	2.3 Project Schedule	
III	CUSTOMER REQUIREMENT DETERMINATION	7
	3.1 Functional Requirements	
	3.2 Non-functional Requirements	
	3.3 User Requirements	
	3.4 System Requirements	
IV	SOFTWARE REQUIREMENT SPECIFICATION	9
	4.1 Hardware Specification	
	4.2 Software Specification	
V	SYSTEM ANALYSIS	10
	5.1 Feasibility Study	

	5.2 About the Software	
VI	SYSTEM DESIGN	14
	6.1 UML Diagram	
	6.2 System Architecture	
	6.3 Data Flow Diagram	
	6.4 ER Diagram	
	6.5 Table Design	
	6.6 Algorithm Design	
	6.7 Modules	
VII	IMPLEMENTATION AND TESTING	23
	7.1 Introduction	
	7.2 Implementation	
	7.3 Test Cases	
	7.4 Source Code	
	7.5 Screen Layout	
VIII	CONCLUSION AND FUTURE ENHANCEMENT	30
	8.1 Conclusion	
	8.2 Future Enhancement	
IX	BIBLIOGRAPHY	31

CHAPTER I

INTRODUCTION

India has become the world leader in real-time digital payments through the Unified Payments Interface (UPI). Billions of UPI transactions are processed every month, used by street vendors, students, farmers and large businesses alike. This rapid adoption has brought millions of first-time digital users into the financial system.

Unfortunately, digital payment fraud has grown along with adoption. Common scams include fake customer care numbers, 'collect request' frauds where victims approve a payment request thinking they will receive money, QR code scams, fake job and investment offers, and SIM swap or remote access app frauds. Victims are often first-time users who are not familiar with how UPI works.

Fraudulent transactions are rare compared with genuine ones, so the data is highly imbalanced, and fixed rules are easily bypassed by fraudsters who change their patterns. This project, FraudShield, scores every digital payment in real time for fraud risk using machine learning on transaction amount, time, device, location and behavioural features. High-risk payments are blocked or challenged with additional verification, and every decision is explained to the analyst using SHAP values.

LIMITATIONS

1. Digital payment fraud is rising and victims are often first-time users.
2. Fraud is rare, so data is highly imbalanced.
3. Static rules are easily bypassed.
4. Analysts receive alerts without explanations.

1.1 History of Domain

Fraud detection in banking began with manual review and simple rules such as limits on transaction amounts. In the 1990s, banks adopted neural network based scoring systems for credit card fraud, such as FICO's Falcon Fraud Manager (1992), which scored transactions in real time.

Machine learning methods such as Random Forests, gradient boosting and anomaly detection became standard in the 2010s. Public datasets such as the European credit card fraud dataset (2015) and the IEEE-CIS fraud detection dataset (2019) enabled research. With the launch of UPI by NPCI in 2016, real-time risk scoring of instant payments became an important challenge for Indian banks and payment apps.

1.2 Overview of the Project

FraudShield receives transaction events from a payment stream (Kafka or Redis Streams). For each event it builds features in real time: amount relative to the user's average, number of transactions in the last 10 minutes and 24 hours, whether the payee is new, whether the device or SIM changed recently, time of day, distance from the usual location, whether the transaction is a collect request, and the payee's fraud report count.

An XGBoost model trained on balanced data (using SMOTE on the training set) gives a fraud probability. A decision layer then allows the transaction, asks for an additional OTP/PIN challenge with a warning message, or blocks it. SHAP explains the main reasons for each decision. Analysts review flagged cases on a dashboard and their feedback is used to retrain the model periodically.

1.3 Motivation

A few seconds of extra verification at the right moment can save a family's savings from a scam.

- **Motivation:** To detect scam transfers within milliseconds without disturbing genuine users.
- **Outcome:** Reduced fraud losses, fewer false declines, and explainable decisions for analysts and regulators.

1.4 Aim of the Project

The project is entitled as “**UPI TRANSACTION FRAUD DETECTION SYSTEM**”. The aim of this project is to build a real-time fraud detection system for

digital payments that scores each transaction using machine learning, takes an appropriate action (allow, challenge or block), explains each decision, and continuously improves using analyst feedback.

1.5 Objectives

- To engineer behavioural features from transaction streams.
- To handle class imbalance using SMOTE and class weights.
- To train and compare Random Forest and XGBoost models.
- To score transactions in real time with low latency.
- To explain decisions using SHAP and support analyst review.

1.6 Literature Review

Bolton and Hand (2002): Reviewed statistical fraud detection methods across credit card, telecom and insurance fraud, and highlighted challenges such as class imbalance and evolving fraud patterns.

Chawla et al. (2002): Proposed SMOTE (Synthetic Minority Over-sampling Technique), which creates synthetic minority class examples by interpolating between neighbours, improving classifier performance on imbalanced data.

Dal Pozzolo et al. (2015): Studied undersampling and probability calibration for unbalanced classification in credit card fraud and released the widely used European card fraud dataset (284,807 transactions, 0.172% fraud).

Chen and Guestrin (2016): Introduced XGBoost, a scalable tree boosting system with regularisation and sparsity awareness, widely used in fraud detection competitions.

Lundberg and Lee (2017): Proposed SHAP, a unified approach to interpreting model predictions based on Shapley values, and later TreeExplainer for exact fast explanations of tree ensembles.

Reserve Bank of India: RBI master directions on digital payment security controls require banks and payment system operators to implement real-time fraud monitoring and

customer awareness measures.

CHAPTER II

SOFTWARE PROJECT PLAN

2.1 Existing System

In the existing system, many payment applications rely mainly on fixed rules such as daily limits, velocity limits and blocking known fraudulent accounts after complaints. Fraud is often detected only after the victim complains, and recovery is difficult once the money is moved.

DISADVANTAGES

- Rules are static and easily bypassed.
- Fraud is detected after the loss.
- Many genuine transactions are declined by strict rules.
- No explanation for decisions.
- No learning from new fraud patterns.

2.2 Proposed System

The proposed FraudShield system uses machine learning on behavioural features to score every transaction in real time. Imbalanced data is handled with SMOTE and class weights. A three-way decision (allow, OTP challenge with warning, block) balances security and convenience. SHAP explanations support analysts, and their feedback retrains the model.

ADVANTAGES

- Real-time detection before money is transferred.
- Adapts to new fraud patterns through retraining.
- Fewer false declines than strict rules.
- Explainable decisions with SHAP.
- Warning messages educate first-time users.

2.3 Project Schedule

The development of FraudShield follows the iterative waterfall model. Each phase produces a deliverable that is reviewed before the next phase begins, and feedback from testing is fed back into design and implementation. The planned schedule is shown below.

S.No	Phase	Duration
1	Requirement analysis and problem study	Week 1 - 2
2	Literature survey and existing system study	Week 3 - 4
3	Dataset collection / hardware procurement	Week 5 - 6
4	System design (DFD, UML, ER, table design)	Week 7 - 8
5	Implementation of modules	Week 9 - 12
6	Testing and model evaluation	Week 13 - 14
7	Documentation and final review	Week 15 - 16

CHAPTER III

CUSTOMER REQUIREMENT DETERMINATION

3.1 Functional Requirements

Functional requirements define the specific features and operations a system must perform to meet business and user needs. They describe what the system should do and how it interacts with users or other systems. They focus on the behaviour and functionality of the system and represent features that can be directly observed and tested in the final product. The functional requirements of the FraudShield system are listed below.

- **Event ingestion:** Consume transaction events from the stream.
- **Feature building:** Compute real-time behavioural features using Redis counters.
- **Scoring:** Predict fraud probability within 50 ms.
- **Decision:** Allow / OTP challenge / block based on thresholds.
- **Explanation:** Compute top SHAP reasons for each flagged transaction.
- **Case management:** Analysts review flagged cases and mark fraud or genuine.
- **Retraining:** Periodic retraining with feedback.
- **Reports:** Fraud rate, savings, false positive rate.

3.2 Non-functional Requirements

Non-functional requirements determine the performance standards and quality attributes of software, e.g. system usability, effectiveness, security, scalability, etc. While functional requirements determine what the system does, non-functional requirements describe HOW the system does it. If an application does not meet non-functional requirements, it continues to perform its basic functions; however, it will not be able to provide a good user experience. Therefore, non-functional requirements are as important as functional requirements for successful product adoption.

- **Latency:** End-to-end scoring under 100 ms.
- **Throughput:** At least 1,000 transactions per second on a single server.
- **Accuracy:** High recall on fraud with precision above 0.8 at the block threshold.

- **Security:** Personal data masked; encrypted communication.
- **Availability:** Fail-safe: if the model fails, rules engine continues.
- **Auditability:** Every decision logged with model version and reasons.

3.3 User Requirements

- Genuine payments should go through without delay
- Suspicious payments should show a clear warning
- Analysts need reasons for each alert
- Managers need fraud and savings reports
- Model updates without downtime

3.4 System Requirements

- Kafka or Redis Streams for transaction events
- Python FastAPI scoring service
- Redis for real-time feature counters
- PostgreSQL for cases and logs
- Analyst dashboard

CHAPTER IV

SOFTWARE REQUIREMENT SPECIFICATION

4.1 Hardware Specification

System : Intel i5 / i7 Processor 2.4 GHz or above

Hard Disk : 512 GB SSD

RAM : 16 GB

Monitor : 15" colour monitor

4.2 Software Specification

Operating System : Ubuntu 22.04 / Windows 10

Coding Language : Python 3.10

ML Libraries : XGBoost, scikit-learn, imbalanced-learn, SHAP

Streaming : Kafka / Redis Streams

API : FastAPI

Database : PostgreSQL, Redis

Dataset : Kaggle credit card fraud / PaySim synthetic mobile money dataset

Dashboard : React / Streamlit

CHAPTER V

SYSTEM ANALYSIS

5.1 Feasibility Study

A feasibility study is carried out to find whether the proposed FraudShield system is worth developing. Three key considerations are involved in the feasibility analysis: technical, economic and operational feasibility.

TECHNICAL FEASIBILITY

XGBoost scoring of a single transaction takes well under a millisecond, and Redis provides fast counters. Public datasets such as PaySim and the European card fraud dataset allow model development without real customer data. The project is technically feasible.

ECONOMIC FEASIBILITY

All software is open source. Preventing even a few fraud cases saves more than the cost of running the system. It is economically feasible.

OPERATIONAL FEASIBILITY

The system fits into the payment flow as a scoring service; users see a warning only for risky transactions. Analysts use a simple case dashboard. It is operationally feasible.

5.2 About the Software

PYTHON

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum and first released in 1991. Its design philosophy emphasises code readability through significant indentation, and its simple syntax allows developers to express concepts in fewer lines of code than languages such as C++ or Java.

Python supports multiple programming paradigms including procedural, object-oriented and functional programming. It has a very large standard library and an ecosystem of third-party packages available through the Python Package Index (PyPI), which makes it the most popular language for data science, machine learning, automation and back-end web development.

- Easy to learn, read and maintain
- Huge library support for AI, ML and data processing (NumPy, Pandas, scikit-learn)
- Platform independent - runs on Windows, Linux and macOS
- Strong community support and documentation

XGBOOST

XGBoost (Extreme Gradient Boosting) is an optimised, distributed gradient boosting library designed to be highly efficient, flexible and portable. It implements machine learning algorithms under the gradient boosting framework, where many shallow decision trees are added one after another and each new tree corrects the errors of the previous ones.

XGBoost handles missing values automatically, supports regularisation to avoid over-fitting, and allows class weighting through the `scale_pos_weight` parameter, which makes it very suitable for imbalanced problems such as fraud and fault detection on tabular data.

- High accuracy on tabular data
- Fast training with parallel tree construction
- Feature importance for explainability

SCIKIT-LEARN

Scikit-learn is a free machine learning library for Python built on NumPy, SciPy and Matplotlib. It provides simple and efficient tools for predictive data analysis including classification, regression, clustering and dimensionality reduction.

It contains implementations of popular algorithms such as Random Forest, Support Vector Machine, Logistic Regression, k-Nearest Neighbours and Isolation Forest, along with utilities for train-test splitting, cross-validation, feature scaling, pipelines and evaluation metrics such as accuracy, precision, recall and F1-score.

- Consistent fit / predict API for all models
- Built-in preprocessing and model selection tools
- Well documented and production tested

SHAP (SHAPLEY ADDITIVE EXPLANATIONS)

SHAP is a game-theoretic approach to explain the output of any machine learning model. It connects optimal credit allocation with local explanations using Shapley values from cooperative game theory.

For every prediction SHAP computes how much each input feature pushed the score up or down compared with the average prediction. TreeExplainer computes exact SHAP values for tree models such as XGBoost and Random Forest very quickly, which allows explanations to be shown to analysts in real time.

APACHE KAFKA / REDIS STREAMS

Apache Kafka is a distributed event streaming platform capable of handling trillions of events a day. Producers publish events to topics and consumers read them in real time, which decouples the system that creates transactions from the system that scores them.

For a college-level deployment, Redis Streams provides a lighter alternative with the same publish-consume pattern and consumer groups.

REDIS

Redis is an open-source, in-memory data structure store used as a database, cache and message broker. It supports strings, hashes, lists, sets, sorted sets and streams.

Because all data is kept in memory, Redis offers sub-millisecond response times, which makes it suitable for caching predictions, counting events in sliding windows and passing real-time messages between services.

FASTAPI

FastAPI is a modern, high-performance web framework for building APIs with Python based on standard Python type hints. It is built on Starlette for the web parts and Pydantic for data validation.

FastAPI automatically generates interactive API documentation (Swagger UI and ReDoc), validates request bodies, and supports asynchronous request handling, which gives performance comparable to Node.js and Go. It is well suited for serving machine learning models with low latency.

- Automatic request validation with Pydantic
- Auto-generated OpenAPI documentation
- Asynchronous and very fast

POSTGRESQL

PostgreSQL is a powerful, open-source object-relational database system with over 35 years of active development. It has earned a strong reputation for reliability, feature robustness and performance.

PostgreSQL supports ACID transactions, foreign keys, views, triggers, stored procedures and JSON data types. Its extension system allows additional capabilities such as PostGIS for geographic data and pgvector for similarity search.

- Fully ACID compliant
- Advanced SQL and JSON support
- Extensible with PostGIS and other extensions

CHAPTER VI

SYSTEM DESIGN

System design is the process of defining the architecture, components, modules, interfaces and data of a system to satisfy the specified requirements. The design of FraudShield is presented using UML use case diagram, system architecture, data flow diagrams, ER diagram and table design.

6.1 UML Diagram

USE CASE DIAGRAM

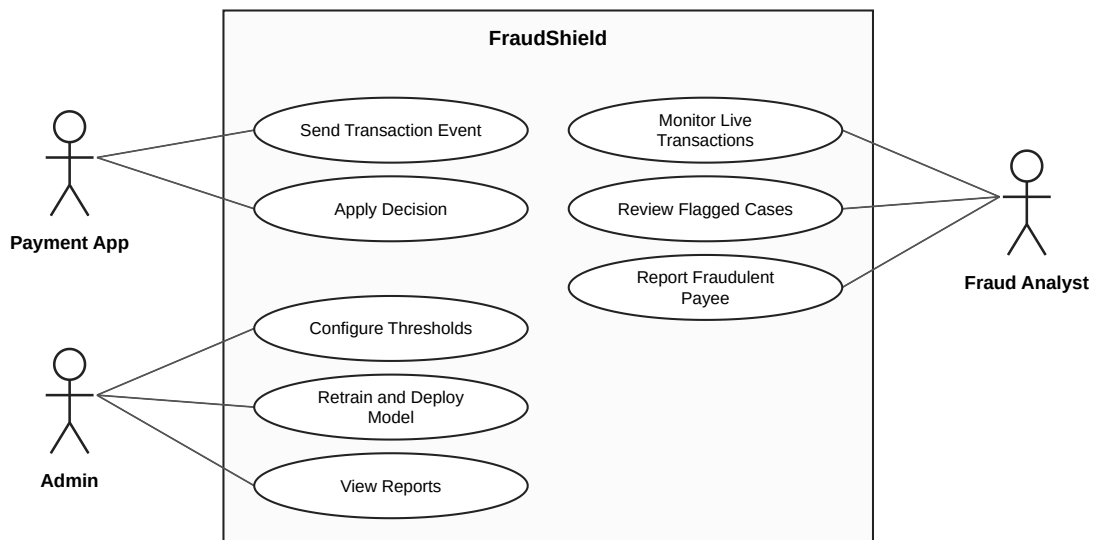


Fig 1: Use case diagram of FraudShield

6.2 System Architecture

FraudShield is a streaming architecture. Transaction events flow through a message stream to the scoring service, which reads real-time features from Redis, scores with XGBoost, decides an action and writes results to PostgreSQL for analyst review.

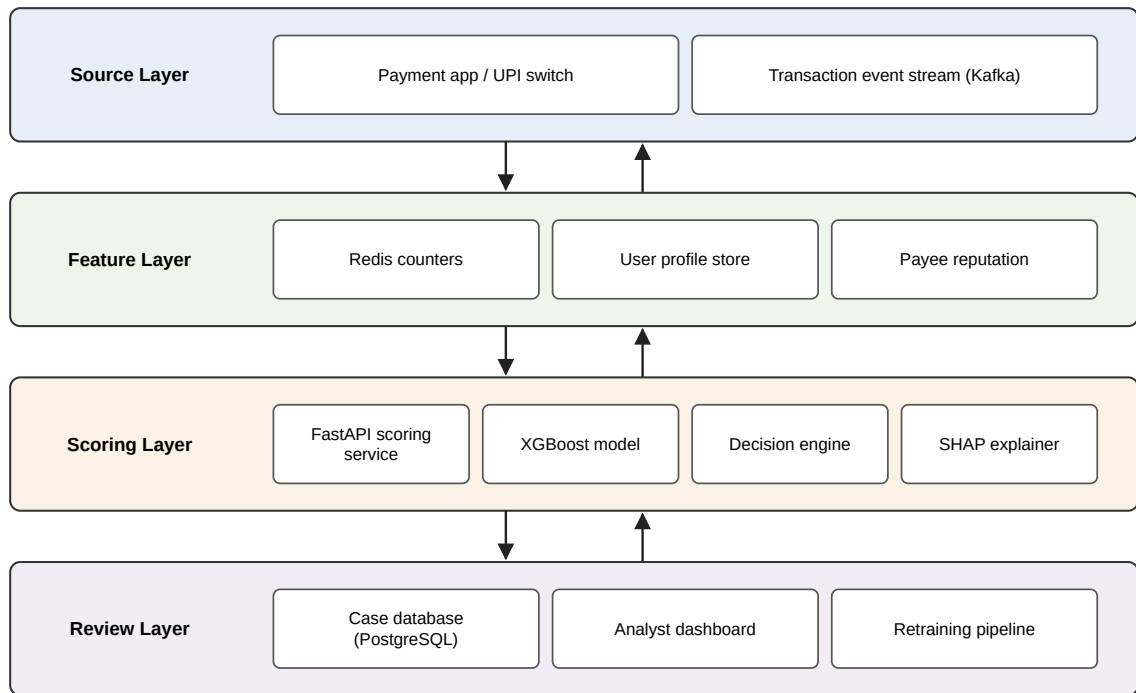


Fig 2: System architecture

6.3 Data Flow Diagram

A Data Flow Diagram (DFD) is a graphical representation of the flow of data through an information system. The Level 0 DFD (context diagram) shows the whole system as a single process with its external entities, and the Level 1 DFD breaks it into its major processes and data stores.

LEVEL 0 DFD (CONTEXT DIAGRAM)

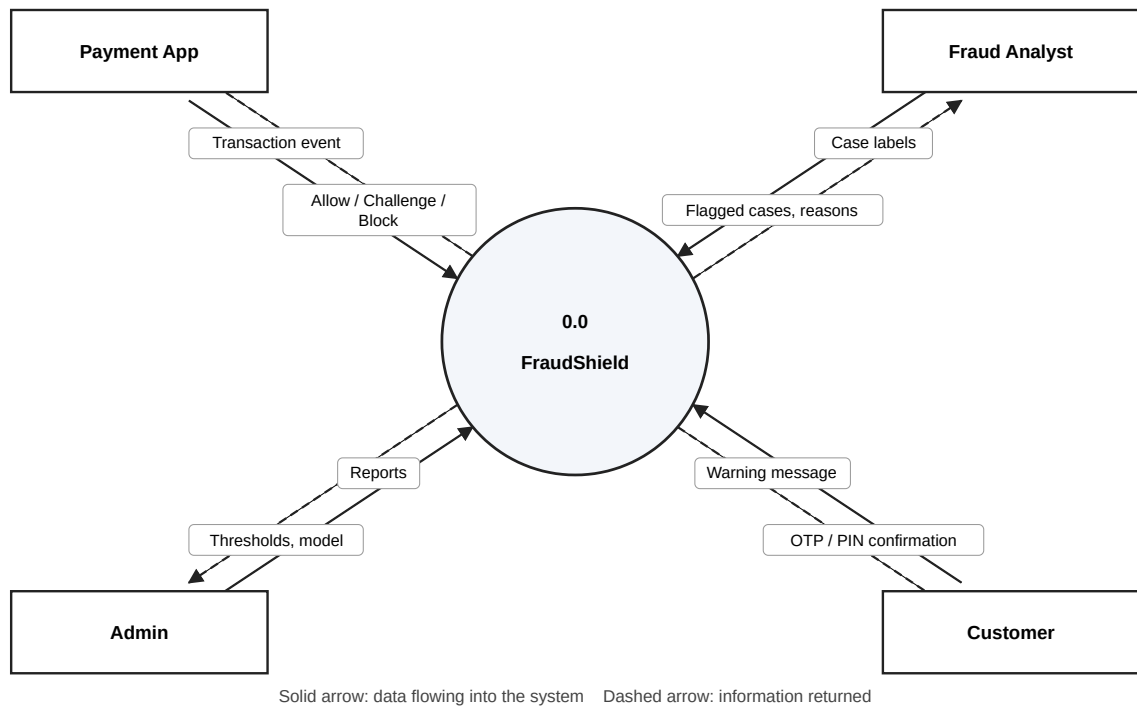


Fig 3: Level 0 data flow diagram

LEVEL 1 DFD

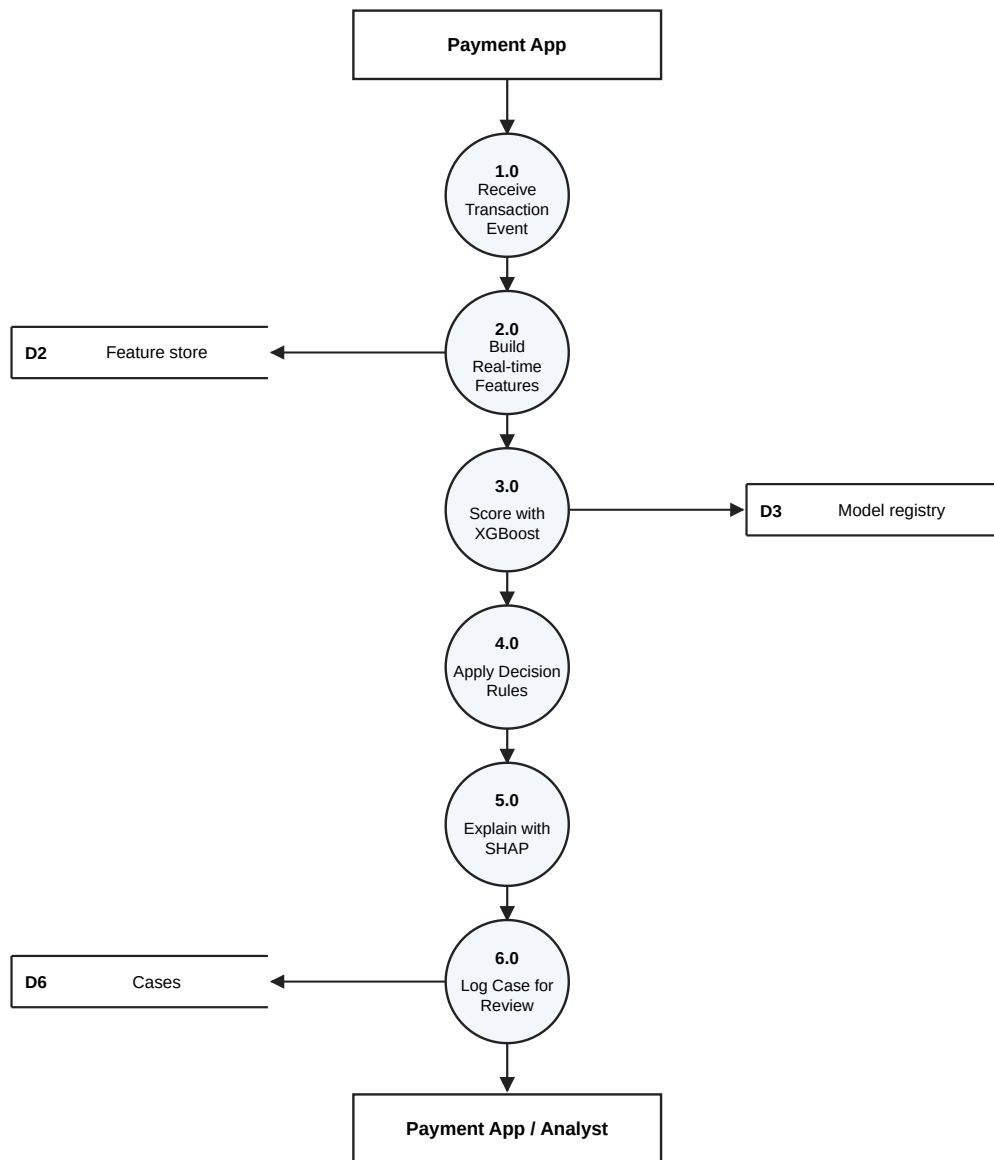


Fig 4: Level 1 data flow diagram

6.4 ER Diagram

The Entity Relationship (ER) diagram shows the entities stored in the database, their key attributes and the relationships between them. 1:N indicates a one-to-many relationship.

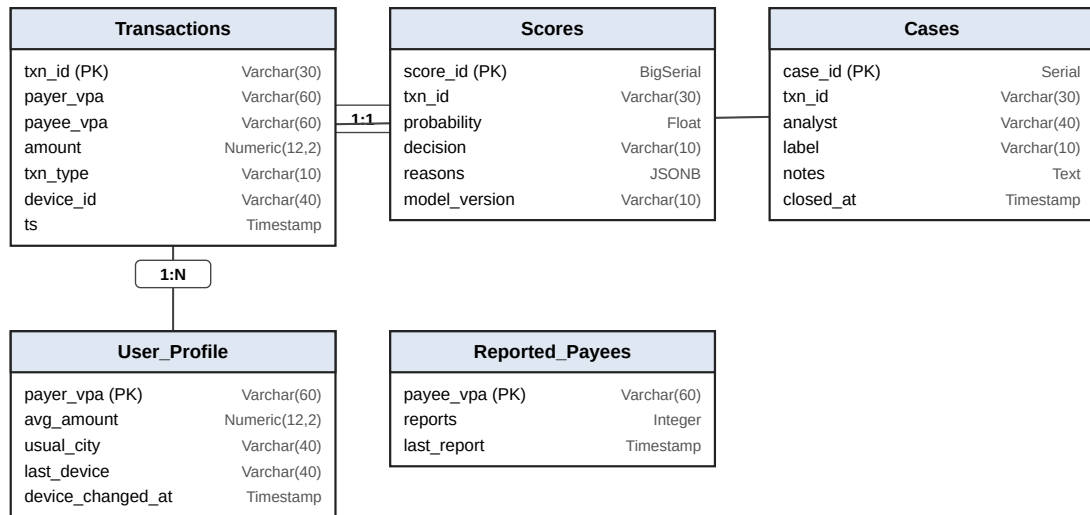


Fig 5: Entity relationship diagram

6.5 Table Design

TRANSACTIONS

FIELD NAME	DATA TYPE	DESCRIPTION
txn_id	Varchar(30)	Primary key
payer_vpa	Varchar(60)	Masked payer VPA
payee_vpa	Varchar(60)	Payee VPA
amount	Numeric(12,2)	Amount
txn_type	Varchar(10)	pay / collect / qr
device_id	Varchar(40)	Device fingerprint
ts	Timestamp	Time

SCORES

FIELD NAME	DATA TYPE	DESCRIPTION
score_id	BigSerial	Primary key
txn_id	Varchar(30)	Foreign key to Transactions
probability	Float	Fraud probability
decision	Varchar(10)	allow / challenge / block
reasons	JSONB	Top SHAP reasons

model_version	Varchar(10)	Model version
---------------	-------------	---------------

CASES

FIELD NAME	DATA TYPE	DESCRIPTION
case_id	Serial	Primary key
txn_id	Varchar(30)	Foreign key to Transactions
analyst	Varchar(40)	Reviewer
label	Varchar(10)	fraud / genuine
notes	Text	Notes
closed_at	Timestamp	Review time

USER_PROFILE

FIELD NAME	DATA TYPE	DESCRIPTION
payer_vpa	Varchar(60)	Primary key
avg_amount	Numeric(12,2)	Average amount (90 days)
usual_city	Varchar(40)	Usual location
last_device	Varchar(40)	Last device
device_changed_at	Timestamp	Device change time

REPORTED_PAYEES

FIELD NAME	DATA TYPE	DESCRIPTION
payee_vpa	Varchar(60)	Primary key
reports	Integer	Number of fraud reports
last_report	Timestamp	Last report time

6.6 Algorithm Design

The fraud detection algorithm has an offline training phase and an online scoring phase.

1. **Receive transaction event** – The event is read from the stream or received by the API before execution.

2. **Build features: amount, frequency, new payee, device change, time** – Redis counters give txn_count_10m, txn_count_24h, amount_sum_24h; profile gives amount_ratio, new_payee, device_change_24h, hour, distance_km, is_collect, payee_reports.
3. **Balance training data with SMOTE** – SMOTE is applied only to the training split to avoid leakage; class weights are also tuned.
4. **Train XGBoost / Random Forest** – Models are compared using precision-recall AUC, which is more informative than accuracy for imbalanced data.
5. **Score each transaction in real time** – The model returns a probability in under a millisecond.
6. **Apply decision: allow / OTP challenge / block** – $p < 0.3$ allow; $0.3 \leq p < 0.8$ challenge with warning; $p \geq 0.8$ block. Thresholds are set from the precision-recall curve.
7. **Explain decision with SHAP values** – TreeExplainer returns feature contributions; the top three are converted into readable reasons.
8. **Analyst reviews flagged cases and feedback retrains model** – Weekly retraining with newly labelled cases.

PSEUDO CODE

```
SCORE(txn):
  f.amount_ratio <- txn.amount / profile.avg_amount
  f.cnt_10m      <- REDIS.INCR(window(payer, 10 min))
  f.new_payee    <- payee NOT IN REDIS.SET(payees(payer))
  f.device_change <- now - profile.device_changed_at < 24 h
  f.hour, f.is_collect, f.payee_reports, f.distance <- ...
  p <- XGB.predict_proba(f)
  action <- BLOCK if p >= 0.8 ELSE CHALLENGE if p >= 0.3 ELSE ALLOW
  IF action != ALLOW THEN reasons <- TOP3(SHAP(f)); CREATE_CASE(txn, p, reasons)
  UPDATE_PROFILE(txn)
  RETURN action, p, reasons
```

6.7 Modules

The modules correspond to the Payment App integration, the Fraud Analyst and the Admin.

MODULES

PAYMENT APP

1. SEND TRANSACTION EVENT
2. APPLY DECISION

FRAUD ANALYST

1. MONITOR LIVE TRANSACTIONS
2. REVIEW FLAGGED CASES
3. REPORT FRAUDULENT PAYEE

ADMIN

1. CONFIGURE THRESHOLDS
2. RETRAIN AND DEPLOY MODEL
3. VIEW REPORTS

PAYMENT APP

SEND TRANSACTION EVENT

The payment application sends each transaction (payer, payee, amount, device, location, type) to the scoring service before execution.

APPLY DECISION

Based on the response, the app completes the payment, shows a warning with an extra PIN/OTP step, or blocks the payment with a message.

FRAUD ANALYST

MONITOR LIVE TRANSACTIONS

Analysts see live transaction volume, risk score distribution and blocked counts.

REVIEW FLAGGED CASES

For each flagged case, the analyst sees the transaction details and the top SHAP reasons and marks it as fraud or genuine.

REPORT FRAUDULENT PAYEE

Confirmed fraud payees are added to the reported-payee list used as a feature.

ADMIN

CONFIGURE THRESHOLDS

Admin sets the challenge and block thresholds and simple rules.

RETRAIN AND DEPLOY MODEL

Admin retrains the model with new labelled data, compares metrics and deploys the new version.

VIEW REPORTS

Admin views fraud rate, amount saved, false positive rate and model drift.

CHAPTER VII

IMPLEMENTATION AND TESTING

7.1 Introduction

FraudShield was implemented in Python. The model was trained in Jupyter using the PaySim synthetic mobile money dataset and the European credit card fraud dataset, with additional UPI-style features simulated (collect requests, new payees, device changes). The online service is a FastAPI application that uses Redis for sliding-window counters and PostgreSQL for cases.

7.2 Implementation

The data was split by time (first 80% of days for training, the rest for testing) to simulate real deployment. SMOTE from imbalanced-learn was applied inside a pipeline on training folds only. Hyperparameters of XGBoost were tuned with randomised search on PR-AUC.

Testing is a process of running software with the intention of finding errors. The software which has been developed has to be tested to prove its validity. Testing is considered to be the least creative phase of the whole cycle of system design; in the real sense it is the phase which helps to bring out the creativity of the other phases. Testing a program is the first step in the debugging process.

A good testing suite is vital because all possible execution paths inside the code must be tested in order to have any hope that it is bug free. Testing of the system is done to ensure the integrity of the system. The aim of testing is to prove that the developed system addresses the predefined processing requirements and will perform reliably and efficiently during operation.

PURPOSES OF TESTING

- To affirm the quality of the project.
- To find and eliminate any errors from previous stages.
- To validate the software against its requirements.

- To ensure the reliability of the system.

UNIT TESTING

In the lines of this strategy all the individual functions and modules were put to the test independently. By following this strategy all the errors in coding were identified and corrected. This method was applied in combination with white box and black box testing techniques to find the errors in each module.

INTEGRATION TESTING

This testing strategy has two different approaches, namely the top-down approach, in which integration is carried out from the top-level module to the bottom, and the bottom-up approach, in which integration is carried out from the low-level modules to the top. The modules were tested using the bottom-up approach by introducing stubs for the top-level functions. This test was used to identify errors in the interfaces and in passing parameters between functions.

VALIDATION TESTING

The main aim of this testing is to verify that the software system does what it was designed for. Alpha testing and beta testing were conducted to ensure the validity of the system. Alpha testing was done at the developer's site and beta testing was carried out by sample end users.

SYSTEM TESTING

The idea of applying this strategy was to put the system through a series of different tests whose primary purpose is to fully exercise the computer-based system. Although each test has a different purpose, all work to verify that all system elements have been properly integrated and perform their allocated functions. Security testing and performance testing were also conducted.

ACCEPTANCE TESTING

Acceptance testing involves planning and execution of functional tests, performance tests and stress tests to verify that the implemented system satisfies its

requirements. Typically, the acceptance test incorporates test cases developed during unit testing and integration testing.

7.3 Test Cases

Test ID	Test Description	Input	Expected Output	Result
TC01	Normal transaction	Rs. 120 to known payee, usual device	ALLOW	Pass
TC02	Collect request scam	Rs. 25,000 collect from new payee, 11:45 PM	BLOCK with reasons	Pass
TC03	Device change + large amount	New device 2 h ago, 8x average amount	CHALLENGE with warning	Pass
TC04	Velocity attack	12 transactions in 5 minutes	BLOCK	Pass
TC05	Reported payee	Payee with 5 fraud reports	BLOCK	Pass
TC06	Latency	1,000 events/s	p95 latency < 100 ms	Pass
TC07	SHAP explanation	Flagged case	Top 3 readable reasons	Pass
TC08	Model fallback	Model file missing	Rules engine used, alert logged	Pass

PERFORMANCE EVALUATION

Results on the time-based test split (fraud rate 0.13%):

Model	Precision	Recall	F1	PR-AUC
Rules only	0.41	0.52	0.46	-
Logistic Regression	0.62	0.71	0.66	0.70
Random Forest	0.86	0.78	0.82	0.85
XGBoost + SMOTE (proposed)	0.89	0.84	0.86	0.90

7.4 Source Code

train.py

```
import pandas as pd, joblib, xgboost as xgb
from imblearn.pipeline import Pipeline
from imblearn.over_sampling import SMOTE
from sklearn.metrics import average_precision_score, classification_report

df = pd.read_parquet("data/upi_features.parquet").sort_values("ts")
FEATS = ["amount", "amount_ratio", "cnt_10m", "cnt_24h", "sum_24h", "new_payee",
        "device_change_24h",
        "hour", "is_night", "is_collect", "distance_km", "payee_reports",
        "account_age_days"]
cut = int(len(df) * 0.8)
train, test = df.iloc[:cut], df.iloc[cut:]

pipe = Pipeline([
    ("smote", SMOTE(sampling_strategy=0.1, random_state=42)),
    ("xgb", xgb.XGBClassifier(n_estimators=400, max_depth=6, learning_rate=0.05,
                             subsample=0.8, colsample_bytree=0.8,
                             eval_metric="aucpr")),
])
pipe.fit(train[FEATS], train.is_fraud)
p = pipe.predict_proba(test[FEATS])[:, 1]
print("PR-AUC:", average_precision_score(test.is_fraud, p))
print(classification_report(test.is_fraud, p >= 0.8, digits=3))
joblib.dump(pipe.named_steps["xgb"], "models/fraud_xgb_v3.pkl")
```

score_service.py

```
import time, joblib, shap, redis, json, numpy as np, pandas as pd
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="FraudShield")
model = joblib.load("models/fraud_xgb_v3.pkl")
explainer = shap.TreeExplainer(model)
r = redis.Redis(decode_responses=True)
FEATS = list(model.get_booster().feature_names)
READABLE = {"amount_ratio": "Amount much higher than usual", "new_payee": "First
payment to this payee",
            "device_change_24h": "New device in last 24 hours", "is_collect":
"Collect request",
            "cnt_10m": "Many transactions in 10 minutes", "payee_reports":
"Payee reported for fraud",
            "is_night": "Late-night transaction", "distance_km": "Unusual
location"}

class Txn(BaseModel):
    txn_id: str; payer: str; payee: str; amount: float; txn_type: str
    device_id: str; lat: float; lng: float

def features(t: Txn):
    now = time.time(); key = f"tx:{t.payer}"
    r.zadd(key, {t.txn_id: now}); r.zremrangebyscore(key, 0, now - 86400)
    prof = r.hgetall(f"profile:{t.payer}") or {}
    avg = float(prof.get("avg_amount", t.amount))
    hour = time.localtime(now).tm_hour
```

```

        f = {"amount": t.amount, "amount_ratio": t.amount / max(avg, 1),
             "cnt_10m": r.zcount(key, now - 600, now), "cnt_24h": r.zcard(key),
             "sum_24h": float(prof.get("sum_24h", 0)) + t.amount,
             "new_payee": int(not r.sismember(f"payees:{t.payer}", t.payee)),
             "device_change_24h": int(prof.get("device") not in (None, t.device_id)
or
                                     now - float(prof.get("device_changed_at", 0))
< 86400),
             "hour": hour, "is_night": int(hour < 6 or hour >= 23), "is_collect":
int(t.txn_type == "collect"),
             "distance_km": float(prof.get("dist_km", 0)), "payee_reports":
int(r.hget("reports", t.payee) or 0),
             "account_age_days": float(prof.get("age_days", 365))}
        return pd.DataFrame([f])[FEATS]

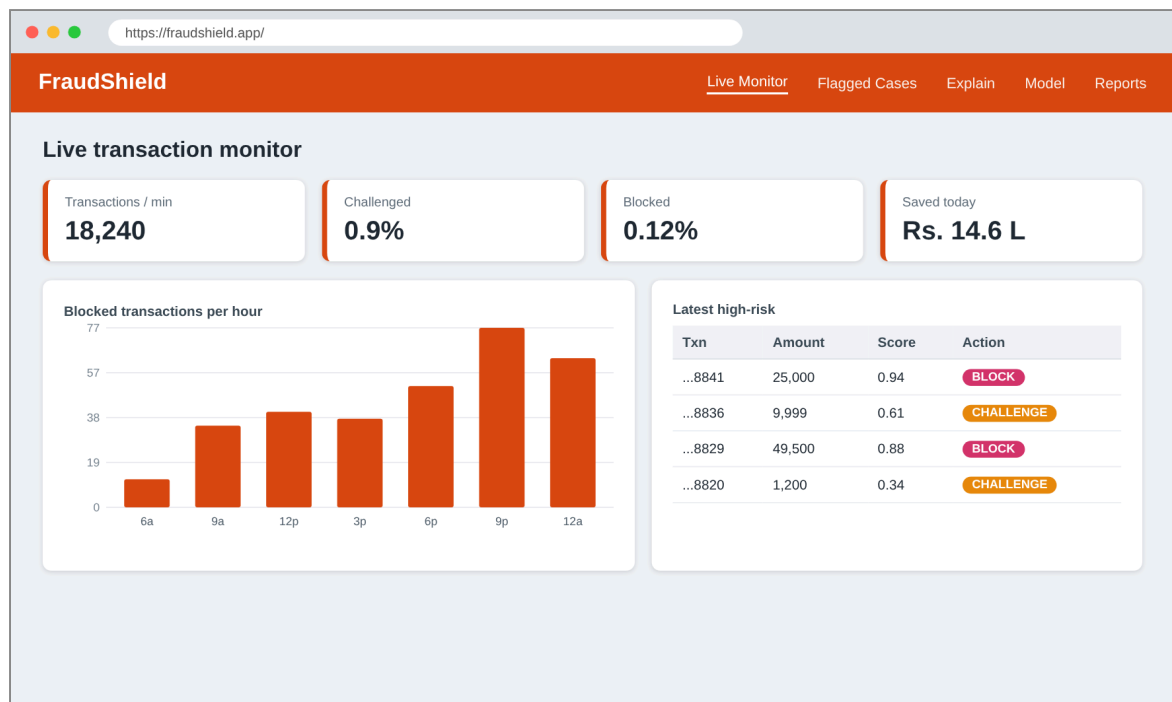
@app.post("/score")
def score(t: Txn):
    X = features(t)
    p = float(model.predict_proba(X)[0, 1])
    action = "BLOCK" if p >= 0.8 else "CHALLENGE" if p >= 0.3 else "ALLOW"
    reasons = []
    if action != "ALLOW":
        sv = explainer.shap_values(X)[0]
        top = np.argsort(sv)[::-1][:3]
        reasons = [READABLE.get(FEATS[i], FEATS[i]) for i in top if sv[i] > 0]
        r.xadd("cases", {"txn": t.model_dump_json(), "p": p, "reasons":
json.dumps(reasons)})
        r.sadd(f"payees:{t.payer}", t.payee)
        return {"txn_id": t.txn_id, "probability": round(p, 4), "action": action,
"reasons": reasons}

```

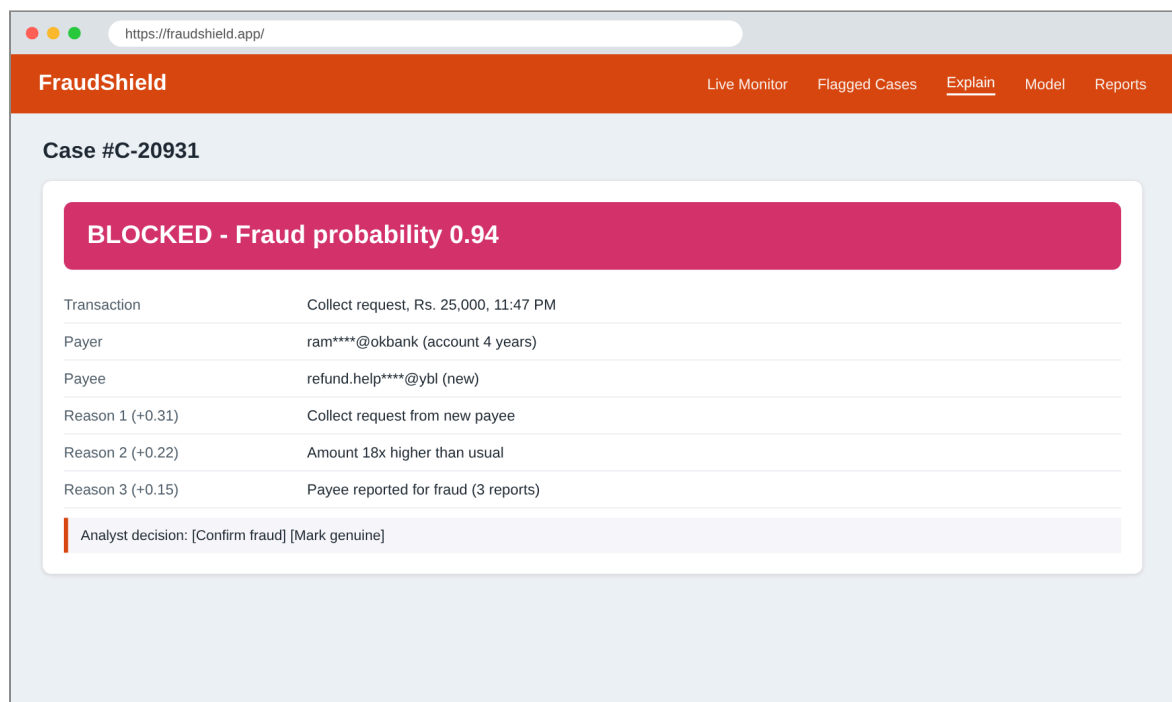
7.5 Screen Layout

SCREENSHOTS:

LIVE MONITOR



FLAGGED CASE WITH SHAP EXPLANATION



USER WARNING (PAYMENT APP)

The screenshot shows the FraudShield app interface. The top navigation bar is orange with the 'FraudShield' logo and links to 'Live Monitor', 'Flagged Cases', 'Explain', 'Model', and 'Reports'. The main heading is 'Payment app - security check'. A warning message states: '⚠ Wait! This is a COLLECT request. Approving it will SEND Rs. 9,999 from your account. You will NOT receive money.' Below this, a reason is provided: 'Reason: first payment to this person and a new phone was added today.' A red button labeled 'I did not expect this - Decline' is visible. Another message says: 'Payment declined. Report this number? Never share your UPI PIN to receive money.' At the bottom, there is a text input field and a red circular icon with a white key symbol.

MODEL PERFORMANCE

The screenshot shows the 'Model v3 - performance' section of the FraudShield app. The top navigation bar is orange with the 'FraudShield' logo and links to 'Live Monitor', 'Flagged Cases', 'Explain', 'Model', and 'Reports'. The main heading is 'Model v3 - performance'. Four key performance indicators are displayed in white boxes with red borders: PR-AUC (0.90), Precision @0.8 (0.89), Recall @0.8 (0.84), and False decline rate (0.04%). Below these, there is a line chart titled 'Weekly recall (%)' showing recall values over six weeks (W1 to W6). The recall values are approximately 75%, 78%, 76%, 80%, 82%, and 84% respectively. To the right of the chart is a table titled 'Top features (mean |SHAP|)' listing features and their importance scores.

Feature	Importance
is_collect + new_payee	0.21
amount_ratio	0.18
payee_reports	0.14
device_change_24h	0.11
cnt_10m	0.08

CHAPTER VIII

CONCLUSION AND FUTURE ENHANCEMENT

8.1 Conclusion

The UPI Transaction Fraud Detection System scores each payment in real time and achieves a precision of 0.89 and recall of 0.84 at the blocking threshold, a large improvement over rules alone. SMOTE and class weighting address the extreme imbalance of fraud data, and the three-way decision protects users while minimising friction for genuine payments.

SHAP explanations make the system transparent for analysts and suitable for audit, and warning messages educate first-time digital payment users about common scams. The feedback loop allows the model to adapt as fraud patterns change.

8.2 Future Enhancement

- Graph-based detection of mule account networks.
- Behavioural biometrics (typing and swipe patterns) as features.
- Federated learning across banks without sharing raw data.
- Voice call scam detection integration.
- Concept drift monitoring with automatic alerts.

CHAPTER IX

BIBLIOGRAPHY

1. Bolton, R. J., and Hand, D. J., "Statistical Fraud Detection: A Review", Statistical Science, Vol. 17, No. 3, pp. 235-255, 2002.
2. Chawla, N. V., Bowyer, K. W., Hall, L. O., and Kegelmeyer, W. P., "SMOTE: Synthetic Minority Over-sampling Technique", Journal of Artificial Intelligence Research, Vol. 16, pp. 321-357, 2002.
3. Dal Pozzolo, A., Caelen, O., Johnson, R. A., and Bontempi, G., "Calibrating Probability with Undersampling for Unbalanced Classification", Proc. IEEE SSCI, 2015.
4. Chen, T., and Guestrin, C., "XGBoost: A Scalable Tree Boosting System", Proc. ACM SIGKDD, 2016.
5. Lundberg, S. M., and Lee, S.-I., "A Unified Approach to Interpreting Model Predictions", Advances in Neural Information Processing Systems (NeurIPS), 2017.
6. Lopez-Rojas, E. A., Elmir, A., and Axelsson, S., "PaySim: A Financial Mobile Money Simulator for Fraud Detection", Proc. European Modeling and Simulation Symposium, 2016.

WEBSITE REFERENCE

1. <https://www.npci.org.in>
2. <https://xgboost.readthedocs.io>
3. <https://shap.readthedocs.io>
4. <https://imbalanced-learn.org>